Spam

In [ ]:
```python
# Import necessary libraries
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB

# Load dataset
# download from https://archive.ics.uci.edu/ml/datasets/SMS+Spam+Collection
data = pd.read_csv("spam.csv", encoding='latin-1')[['v1', 'v2']]
data.columns = ['label', 'message']

# Encode labels: spam = 1, ham = 0
data['label'] = data['label'].map({'spam': 1, 'ham': 0})

# Split dataset into training and test sets
X_train, X_test, y_train, y_test = train_test_split(data['message'], data['lab

# Convert text data into TF-IDF features
vectorizer = TfidfVectorizer(stop_words='english', max_features=1000)
X_train_tfidf = vectorizer.fit_transform(X_train)
X_test_tfidf = vectorizer.transform(X_test)

# Train Naive Bayes model
model = MultinomialNB(alpha=1.0)
model.fit(X_train_tfidf, y_train)

# User input for prediction
user_message = input("Enter an email message: ")

# Transform the input message using the same vectorizer
user_message_tfidf = vectorizer.transform([user_message])

# Predict whether spam or not
prediction = model.predict(user_message_tfidf)

# Display output
if prediction[0] == 1:
    print("This message is classified as: SPAM")
else:
    print("This message is classified as: NOT SPAM")
```

```
Enter an email message: send the otp generated
This message is classified as: NOT SPAM
```

Churn

In [ ]:
```python
import pandas as pd
```

```
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.naive_bayes import GaussianNB

# Load your uploaded dataset
data = pd.read_csv("churn.csv")

# Drop customerID (non-predictive)
if 'customerID' in data.columns:
    data = data.drop('customerID', axis=1)

# Clean and preprocess
data['TotalCharges'] = pd.to_numeric(data['TotalCharges'], errors='coerce')
# Select only numeric columns for mean calculation
numeric_cols = data.select_dtypes(include=np.number).columns
data[numeric_cols] = data[numeric_cols].fillna(data[numeric_cols].mean())

# Separate features and label
X = data.drop('Churn', axis=1)
y = data['Churn']

# Encode categorical features
label_encoders = {}
for col in X.columns:
    if X[col].dtype == 'object':
        le = LabelEncoder()
        X[col] = le.fit_transform(X[col])
        label_encoders[col] = le

# Encode target
target_encoder = LabelEncoder()
y = target_encoder.fit_transform(y)

# Split data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, randc

# Scale numeric features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)

# Train Naive Bayes model
model = GaussianNB()
model.fit(X_train_scaled, y_train)

# Collect user input
print("Enter new customer details to predict churn:")

input_data = {}
for col in X.columns:
    if col in label_encoders:
        options = list(label_encoders[col].classes_)
        print(f"{col} options: {options}")
```

```
        val = input(f"Enter {col}: ")
        # Handle unseen input
        if val not in label_encoders[col].classes_:
            label_encoders[col].classes_ = np.append(label_encoders[col].class
        input_data[col] = label_encoders[col].transform([val])[0]
    else:
        val = float(input(f"Enter {col} (numeric): "))
        input_data[col] = val

# Convert to DataFrame
user_df = pd.DataFrame([input_data])

# Scale numeric features
user_scaled = scaler.transform(user_df)

# Predict churn
pred = model.predict(user_scaled)
if pred[0] == 1:
    print("\nPrediction: The customer is likely to CHURN.")
else:
    print("\nPrediction: The customer is NOT likely to churn.")
```

```
Enter new customer details to predict churn:
gender options: ['Female', 'Male']
Enter gender: Male
Enter SeniorCitizen (numeric): 0
Partner options: ['No', 'Yes']
Enter Partner: Yes
Dependents options: ['No', 'Yes']
Enter Dependents: 2
Enter tenure (numeric): 4
PhoneService options: ['No', 'Yes']
Enter PhoneService: 7987954654
MultipleLines options: ['No', 'No phone service', 'Yes']
Enter MultipleLines: Yes
InternetService options: ['DSL', 'Fiber optic', 'No']
Enter InternetService: DSL
OnlineSecurity options: ['No', 'No internet service', 'Yes']
Enter OnlineSecurity: No
OnlineBackup options: ['No', 'No internet service', 'Yes']
Enter OnlineBackup: Yes
DeviceProtection options: ['No', 'No internet service', 'Yes']
Enter DeviceProtection: No
TechSupport options: ['No', 'No internet service', 'Yes']
Enter TechSupport: No
StreamingTV options: ['No', 'No internet service', 'Yes']
Enter StreamingTV: Yes
StreamingMovies options: ['No', 'No internet service', 'Yes']
Enter StreamingMovies: Yes
Contract options: ['Month-to-month', 'One year', 'Two year']
Enter Contract: Two Year
PaperlessBilling options: ['No', 'Yes']
Enter PaperlessBilling: No
PaymentMethod options: ['Bank transfer (automatic)', 'Credit card (automatic)',
'Electronic check', 'Mailed check']
Enter PaymentMethod: Mailed check
Enter MonthlyCharges (numeric): 1000
Enter TotalCharges (numeric): 1500

Prediction: The customer is NOT likely to churn.
```

Movie Review

In [ ]:

```python
# Import libraries
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB

# Load dataset (safe reading)
# Uploaded file path
data = pd.read_csv("movies.csv", encoding='utf-8', on_bad_lines='skip', engine

# Check first few rows
```

```
print(data.head())

# Rename columns if needed
# Ensure your dataset has 'review' and 'sentiment' columns; adjust if they dif
if 'review' not in data.columns or 'sentiment' not in data.columns:
    data.columns = ['review', 'sentiment']

# Split data
X_train, X_test, y_train, y_test = train_test_split(
    data['review'], data['sentiment'], test_size=0.2, random_state=42
)

# Vectorize text
vectorizer = TfidfVectorizer(stop_words='english', max_features=5000)
X_train_tfidf = vectorizer.fit_transform(X_train)

# Train Naive Bayes model
model = MultinomialNB()
model.fit(X_train_tfidf, y_train)

# User input
review = input("Enter a movie review: ")

# Transform and predict
review_tfidf = vectorizer.transform([review])
prediction = model.predict(review_tfidf)

print("Predicted Sentiment:", prediction[0].upper())
```

```
                                              review sentiment
0  One of the other reviewers has mentioned that ...  positive
1  A wonderful little production. <br /><br />The...  positive
2  I thought this was a wonderful way to spend ti...  positive
3  Basically there's a family where a little boy ...  negative
4  Petter Mattei's "Love in the Time of Money" is...  positive
Enter a movie review: I absolutely loved Coolie! For me, this is a 5-star film
all the way. To be honest, it felt more like a Lokesh Kanagaraj movie than a ty
pical Rajini sir mass entertainer, but that was part of the magic. Lokesh’s sig
nature style, the layered screenplay, and all the twists and turns kept me hook
ed from start to finish.
Predicted Sentiment: POSITIVE
```

Diabetes

In [ ]:

```
# Import libraries
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.naive_bayes import GaussianNB

# Load dataset
# Dataset: https://www.kaggle.com/datasets/uciml/pima-indians-diabetes-databas
```

```
data = pd.read_csv("diabetes.csv")

# Split data
X = data.drop('Outcome', axis=1)
y = data['Outcome']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, randc

# Scale data
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)

# Train Naive Bayes
model = GaussianNB()
model.fit(X_train_scaled, y_train)

# User input
pregnancies = int(input("Enter number of pregnancies: "))
glucose = float(input("Enter glucose level: "))
bp = float(input("Enter blood pressure: "))
skin = float(input("Enter skin thickness: "))
insulin = float(input("Enter insulin level: "))
bmi = float(input("Enter BMI: "))
dpf = float(input("Enter Diabetes Pedigree Function: "))
age = int(input("Enter age: "))

# Prepare input
user_input = pd.DataFrame([[pregnancies, glucose, bp, skin, insulin, bmi, dpf,
                            columns=X.columns)
user_input_scaled = scaler.transform(user_input)

# Predict
prediction = model.predict(user_input_scaled)
if prediction[0] == 1:
    print("The patient is likely to have the disease.")
else:
    print("The patient is NOT likely to have the disease.")
```

```
Enter number of pregnancies: 2
Enter glucose level: 110
Enter blood pressure: 72
Enter skin thickness: 4
Enter insulin level: 115
Enter BMI: 28
Enter Diabetes Pedigree Function: 2
Enter age: 20
The patient is likely to have the disease.
```

In [ ]:

```
import numpy as np
from sklearn.naive_bayes import GaussianNB
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# 1. Generate a synthetic dataset
# We'll create a simple dataset with two features and two classes
```

```
X = np.array([
    [1, 2], [1.5, 1.8], [5, 8], [8, 8], [1, 0.6],
    [9, 11], [8, 2], [10, 2], [9, 3], [5, 6]
])
y = np.array([0, 0, 1, 1, 0, 1, 0, 1, 1, 1]) # 0 for class A, 1 for class B

# 2. Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, randc

# 3. Initialize the Naive Bayes classifier (Gaussian Naive Bayes for continuou
model = GaussianNB()

# 4. Train the model using the training data
model.fit(X_train, y_train)

# 5. Make predictions on the test data
y_pred = model.predict(X_test)

# 6. Evaluate the model's performance
accuracy = accuracy_score(y_test, y_pred)
print(f"Predicted classes for test data: {y_pred}")
print(f"Actual classes for test data:   {y_test}")
print(f"Accuracy of the Naive Bayes model: {accuracy:.2f}")

# 7. Predict for a new, unseen data point
new_data_point = np.array([[2, 2.5]])
prediction_new = model.predict(new_data_point)
print(f"Prediction for new data point {new_data_point[0]}: Class {prediction_r
```

Generate Data: A small, synthetic dataset X (features) and y (labels) is created to represent a simple classification problem.

Split Data: The dataset is divided into training and testing sets using train_test_split to evaluate the model's generalization ability on unseen data.

Initialize Model: GaussianNB is chosen because the synthetic features are continuous, and this variant of Naive Bayes assumes a Gaussian distribution for the features within each class. Other variants like MultinomialNB (for count data) or BernoulliNB (for binary data) exist for different data types.

Train Model: The fit method trains the Naive Bayes model by calculating the prior probabilities of each class and the likelihoods of features given each class.

Make Predictions: The predict method uses the trained model to classify new data points in the test set.

Evaluate: The accuracy_score metric is used to compare the model's predictions (y_pred) with the actual labels (y_test) in the test set.

Predict New Data: A new, unseen data point is provided to demonstrate how to use the trained model for making predictions on individual instances.